

Sintetički skladišni dataset — upute za postavljanje

Generiranje označenog skladišnog dataseta (RGB + ROMB semantika + instance + COCO panoptic) iz UnrealZoo/UnrealCV scene **Storagehouse**, za percepciju autonomnog viličara.

Ideja: Linux + NVIDIA GPU → gotov UnrealZoo UE5.6 binary pokrećeš headless u Podman kontejneru → spajaš se UnrealCV Python klijentom → skripte proizvode označene slike. Kroz scenu letiš s laptopa da odabereš dobre pozicije kamere, pa skripta izrenda cijeli dataset.

1. Preduvjeti

- Linux host s **NVIDIA GPU-om** (≥8 GB VRAM) i driverom (`nvidia-smi` radi).
- **Podman** (ili Docker) + NVIDIA Container Toolkit.
- ~150 GB slobodnog diska (UE paket je ~67 GB). Koristi veliki disk, ne sistemski.
- Pristup internetu za GitHub i ModelScope/HuggingFace (skidanje paketa).
- Laptop s Pythonom (za interaktivni flythrough preko SSH-a). Rendering ide na GPU hostu; laptop ga samo upravlja.

2. Jednokratno postavljanje

GPU u kontejner (CDI):

```
mkdir -p ~/.config/cdi-unreal
sudo nvidia-ctk cdi generate --output=$HOME/.config/cdi-unreal/nvidia.yaml
```

Slika kontejnera — napravi `Containerfile`:

```
FROM nvcr.io/nvidia/cuda:13.2.1-base-ubuntu24.04
ENV DEBIAN_FRONTEND=noninteractive NVIDIA_DRIVER_CAPABILITIES=all NVIDIA_VISIBLE_DEVICES=all
ENV XDG_RUNTIME_DIR=/tmp/xdg-runtime
RUN apt-get update && apt-get install -y --no-install-recommends \
    libvulkan1 vulkan-tools xvfb libx11-6 libxext6 libxrandr2 libxcb1 libxcb-randr0 \
    libxkbcommon0 libxi6 libxcursor1 libxinerama1 libssl2-2.0.0 libgl1 libglu1-mesa \
    libasound2t64 libpulse0 libglib2.0-0t64 ca-certificates curl unzip python3 python3-pip socat \
    && rm -rf /var/lib/apt/lists/* && mkdir -p /tmp/xdg-runtime && chmod 700 /tmp/xdg-runtime
RUN pip3 install --no-cache-dir --break-system-packages unrealcv pillow numpy
WORKDIR /workspace
```

```
podman build -t unreal-cv:latest -f Containerfile .
```

`libglib2.0-0t64` i `socat` su namjerno uključeni — bez `glib-a` `import unrealcv` (koji povlači `cv2`) puca, a `socat` treba za interaktivni flythrough. Ako pokrećeš iz golog imagea, oboje instaliraj u kontejner prije korištenja.

Skini UnrealZoo (UE5.6 Linux paket, ~67 GB) s linkova na <https://github.com/UnrealZoo/unrealzoo-gym> i raspakiraj u radni folder, npr.:

```
/mnt/<disk>/<user>/UnrealEnv/UnrealZoo_UE5_6_Linux/Linux/UnrealZoo_UE5_6.sh
```

Taj `UnrealEnv` folder montira se u kontejner kao `/data`.

3. Pokretanje simulatora (headless)

Bitno: Vulkan treba display → pokreni pod `xvfb-run`. UnrealCV sluša na portu 9104 ali servira preko **unix socket**a (`/tmp/unrealcv_9104.socket`), koji se stvori tek nakon prvog TCP “dodira”. Scena se diže **5-15 min**. Binary pokreni **interaktivno** (`-it`), ne detached. Scenu učitaj nakon dizanja (ime foldera kao argument ne radi).

Prozor A — pokreni Unreal (ostavi otvoreno):

```
podman rm -f uz-scene 2>/dev/null
podman run --rm -it --name uz-scene --userns=keep-id \
  --cdi-spec-dir=$HOME/.config/cdi-unreal --device nvidia.com/gpu=1 \
  -v /mnt/<disk>/<user>/UnrealEnv:/data -e HOME=/tmp -p 9104:9104 \
  unreal-cv:latest \
  bash -c "xvfb-run -a /data/UnrealZoo_UE5_6_Linux/Linux/UnrealZoo_UE5_6.sh -RenderOffscreen -stdout -unattended -
ResX=1920 -ResY=1080 2>&1 | tail -80"
```

Prozor B — čekaj, probudi socket, učitaj scenu:

```
until nvidia-smi | grep -qi unreal; do sleep 15; echo "...dize se"; done; echo RADI
podman exec uz-scene python3 -c "import socket; socket.create_connection(('127.0.0.1',9104),timeout=5).close();
print('dodir ok')"
# učitaj scenu (ako veza pukne tijekom učitavanja, pokreni ovu liniju opet da potvrdiš):
podman exec uz-scene python3 -c "from unrealcv import Client; c=Client('/tmp/unrealcv_9104.socket','unix');
c.connect(); print(c.request('vget /level/name')); c.request('vset /action/game/level Storagehouse', timeout=180)"
# provjera:
podman exec uz-scene python3 -c "from unrealcv import Client; c=Client('/tmp/unrealcv_9104.socket','unix');
c.connect(); print(c.request('vget /unrealcv/status'))"
```

4. Generiranje dataseta

Kopiraj skripte u UnrealEnv folder (= /data). Pipeline ima četiri koraka.

(a) Snimi dobre pozicije kamere (laptop). Leti kroz scenu i stisni SPACE na dobrim mjestima; cilj ~50–60 waypointsa kroz sve prolaze. Rendering je na serveru, pa koristi mali most + SSH tunel.

```
# server (host): izloži host:9105 -> UDS kontejnera, ostavi otvoreno
python3 uds_bridge.py
# laptop (drugi terminal): tunel, ostavi otvoreno
ssh -N -L 9105:127.0.0.1:9105 <user>@<server>
# laptop: snimanje (treba pip install pygame pillow numpy unrealcv)
python3 record_waypoints.py 127.0.0.1 9105      # SPACE = snimi, ENTER = spremi -> waypoints.json
```

Prebaci waypoints.json u serverov UnrealEnv folder.

(b) Izrenda dataset (server, u pozadini).

```
podman exec -d uz-scene bash -c "cd /data && nohup python3 capture_dataset.py > cap500.log 2>&1"
tail -f /mnt/<disk>/<user>/UnrealEnv/cap500.log
```

capture_dataset.py varira osvjetljenje i kut objektiva, po varijanti skriva nasumičan podskup cargo/paleta, i zadržava samo dobre kadrove → /data/dataset_500/ .

(c) Pregled i čišćenje.

```
podman exec uz-scene python3 /data/contact_sheet.py /data/dataset_500      # pregledne stranice
podman exec -e DATASET=/data/dataset_500 uz-scene python3 /data/clean_void.py review 0.03
# pogledaj void_candidates.png, pa obriši one koje potvrdiš:
podman exec -e DATASET=/data/dataset_500 uz-scene python3 /data/clean_void.py drop <id ...>
```

(d) Složi u ROMB format.

```
podman exec uz-scene python3 /data/finalize_dataset.py
# -> /data/dataset_romb/1080p/ (rgb/ labels/ romb.yaml train.txt val.txt + instance/ panoptic/)
```

Skidanje:

```
scp -0 -r <user>@<server>:/mnt/<disk>/<user>/UnrealEnv/dataset_romb ~/Desktop/
```

5. Skripte

Datoteka	Uloga
<code>romb_export.py</code>	Jezgra: iz <code>object_mask</code> rendera gradi ROMB semantiku + instance; drži shemu klasa, paletu i pravila UE-mesh → klasa.
<code>coco_panoptic.py</code>	Izvoz COCO panoptic (id-kodiran PNG + panoptic.json).
<code>record_waypoints.py</code>	Laptop: leti scenom i snima pozicije kamere → waypoints.json.
<code>capture_dataset.py</code>	Glavni generator: randomizacija osvjetljenja/FOV-a, skrivanje tereta, filtri → dataset_500.
<code>clean_void.py</code>	Pregled/brisanje preostalih "void" kadrova; čuva JSON konzistentnim.
<code>finalize_dataset.py</code>	Presloži u ROMB format + train/val split + README.
<code>contact_sheet.py</code>	Složi RGB-ove u pregledne stranice.
<code>viz_panoptic.py</code>	Pretvori panoptic PNG-ove u čitljive slike po instanci / po klasi.
<code>uds_bridge.py</code>	Server: TCP↔UDS most za flythrough s laptopa.
<code>flythrough_recorder.py</code>	Alternativni snimač koji bilježi kontinuiranu putanju kamere.

ROMB klase (0-10 + ignore=11): `drivable`, `person`, `pallet-face`, `pallet-empty`, `pallet-full`, `cargo`, `vehicle-forklift`, `vehicle-other`, `ego-forklift`, `other-object`, `vertical`. Mapiranje UE→ROMB je u `romb_export.py` (`CLASSIFY_RULES`); nova scena = dopiši pravila ondje.

6. Troubleshooting

Simptom	Rješenje
podman ps prazan / samo <code>xvfb-run</code>	Pokreni interaktivno (<code>-it</code>), ne <code>-d</code> .
<code>nvidia-smi</code> ne pokazuje unreal proces	Još se diže (5-15 min); ili je pao (vidi <code>Saved/Logs</code>).
<code>Connection refused</code> na 9104	Socket još ne postoji → TCP dodir + čekaj scenu.
Scenu treba učitati dvaput	Normalno: prvo učitavanje prekine socket; drugi poziv potvrdi.
<code>import cv2 ... libgthread-2.0.so.0</code>	Instaliraj <code>libglb2.0-0t64</code> u kontejner.
<code>socat ... not found</code> (most)	Instaliraj <code>socat</code> u kontejner.
Sve slike identične	Koristiš kameru 0 → koristi kameru 1 (FusionCam).
Maska prazna / jednolična	Prije maske: <code>vset /viewmode object_mask</code> , pa natrag <code>lit</code> za RGB.
<code>AF_INET address must be tuple</code>	UDS: <code>Client('/tmp/unrealcv_9104.socket', 'unix')</code> (drugi arg <code>'unix'</code>).
TCP klijent s laptopa visi na 9104	UnrealCV servira samo UDS → koristi <code>uds_bridge.py</code> + tunel.
Scena pretamna	<code>vrn r.DefaultFeature.AutoExposure.Bias 2</code> (generator to već varira).

Gašenje: `podman rm -f uz-scene` (oslobodi GPU; dataset i skripte ostaju na disku).